

# 망분리를 풀면, 신뢰를 무엇으로 대체하는가

MLS 보상통제를 코드로. 한 요청이 신원·세분화·인가·암호화·탐지를 통과해야 데이터에 닿고, 막는다는 사실을 매번 라이브로 재현한다.

|          |                        |             |                             |
|----------|------------------------|-------------|-----------------------------|
| coverage | <div><div></div></div> | 77.5%       | 31/40 — 검증가능-as-code        |
| verify   | <div><div></div></div> | 21/21       | 라이브, 전 계층 enforcement       |
| cedar    | <div><div></div></div> | 8/8 · 17/17 | 인가 + 에이전트 위임 (반증가능)         |
| rebac    | <div><div></div></div> | 11/11       | OpenFGA 관계 테스트              |
| checkov  | <div><div></div></div> | 452/0       | shift-left · 실 CVE 1건 포착→수정 |

// 21/21은 기능 회귀 스위트(분모 없는 측정 아님) · 갭 22.5%(CONFIGURED 5·GOVERNANCE 2·NOT-COVERED 4)는 행 단위 공개.

identity → segmentation → authz → encrypt → runtime

같은 경로·다른 신원 · alice GET acct-alice 200 bob 같은 경로 403 db /bin/sh 137 · id 137 (zero-exec)

"안전하다"가 아니라 — 갭을 공개한다.

재현 증거 감사자가 부작용 없는 dry-run으로 직접 거부/허용을 확인.

명시된 갭 미검증 22.5%를 행 단위로 공개 — 두루뭉술하지 않다.

정직성 요청자 JWT 행을 증명 전엔 CONFIGURED로 낮춰 뒀다가 라이브로 강제를 증명해 VERIFIED로 올렸다 — 편한 숫자가 아니라 참값.

읽었으면 — 직접 다시 만들어 본다.

시작점 M0 · Cedar 인가 — 빈 파일에서 정책을 재구현해 채점기 11/11. 클러스터 불필요, 노트북에서 python만.

첫 채점 클론 후 한 번만 python -m venv .venv → .venv\Scripts\python.exe -m pip install -r requirements-dev.txt. 그 뒤 .venv\Scripts\python.exe labs\m0\grade.py → 빈 정책도 5/8(default-deny로 Deny가 공짜)에서 출발하는 게 첫 교훈. 첫 PASS ~5분.

트랙 무클러스터 M0·M1·M6·M7은 Codespaces 0설치로 즉시 채점 · 클러스터 M2-M5·M8·M9·M10·M11은 한 세션(RAM 6-10GB). 코어 7 + 심화 5.

---

repo

실습 시작 - 재구현 트랙 (M0부터)

codespaces - 무설치 체험

교육·포트폴리오 레퍼런스 - 법률/금융 자문도 공식 FSC 컴플라이언스 매핑도 아님 · 규제 세부 1차 출처 대조 · MIT